C-SHELL

A Minimal Unix Shell, Built From Scratch in C

Project window	September 23 – October 11, 2026 (19 build days)
Evaluation	October 12, 2026
Language / environment	C · Linux / POSIX
Governing rule	You implement. Claude teaches, reviews, and debugs. No copied implementations.

“Understand the mechanism well enough that you can explain why your implementation works — not just that it runs.”

WHAT CHANGED FROM THE FIRST DRAFT

- Added a dedicated **buffer/catch-up day (Day 16, Oct 8)** before integration testing — the original plan had zero slack across 19 days, and pipes/signals are exactly where students lose 1–2 days.
- Kept the architecture refactor on **Day 5** but flagged in that day's checkpoint that the structure may need a second pass once file descriptors and pipes (Day 6–10) reveal what shape the I/O code actually needs — so you don't over-commit to a module layout too early.
- Split **Level 3 stretch goals** into an explicit priority order instead of a flat list, so a time-crunched week 3 has a clear cut line.
- Added a one-page **Risk & Cut-Line table** (Section 3) so you decide today what gets dropped if you fall behind — not in a panic on Oct 10.
- Added a **pre-eval self-test script** outline (Appendix C) so Day 19 rehearsal is objective, not vibes-based.

Contents

1	What you're building & the mental model	
2	Feature scope — Level 1 / 2 / 3	
3	Risk map & cut-line — what to drop if you fall behind	
4	Daily plan at a glance (19 days)	
5	Day-by-day guide	Day 1 (Sep 23) → Day 19 (Oct 11)
6	AI collaboration rules for this project	
7	Debugging protocol	
8	Reference library	
App. A	Learning notebook template	
App. B	Full integration test matrix	
App. C	Pre-evaluation self-test script	

How to read this guide

```
STUDY FIRST  →  BUILD  →  CHECKPOINT  →  WRITE DOWN WHAT YOU LEARNED

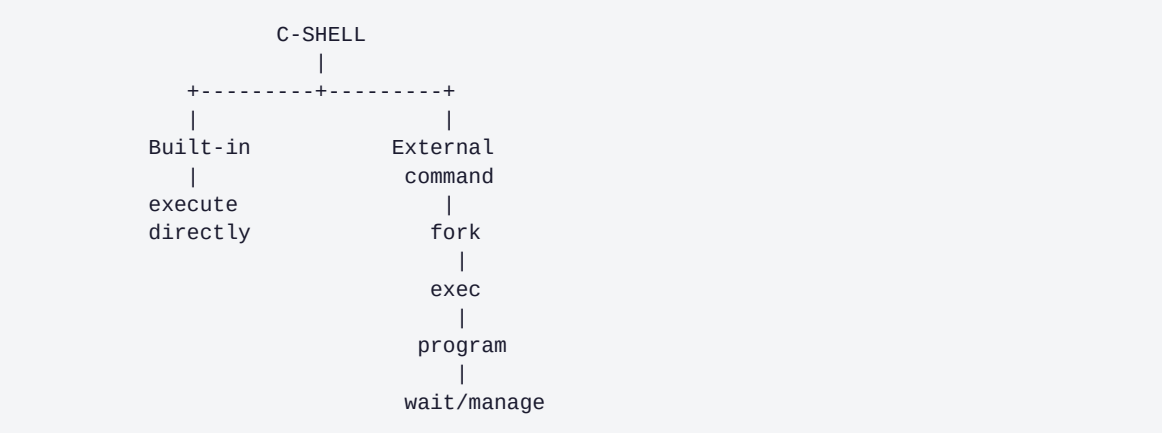
Every day follows this loop. Do not skip the checkpoint — it is the
part that turns "it compiled" into "I can defend this on Oct 12."
```

1 · What You're Building

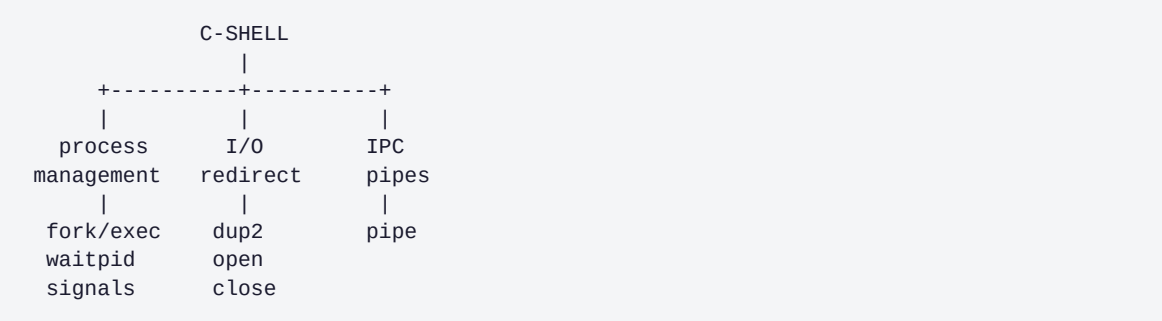
A program called `csh`. It reads a prompt, runs commands, and grows through the project into a shell that handles redirection, pipes, backgrounding, and signals.

```
$ ./csh
csh> ls
csh> ls | grep ".c"
csh> ls > files.txt
csh> sleep 20 &
csh>                          <- returns immediately, sleep keeps running
```

Core execution model



As mechanisms are added:



The chain you must be able to draw from memory

```
User "ls -l"
   |
   v
 Shell --parse--> command + arguments
   |
  fork()
   +------------------+
   |                  |
  child              parent
   |                  |
 execvp()          waitpid()
   |                  |
  ls (runs)      <----+ (waits, then re-prompts)
```

Three APIs, three jobs — memorize this split:

API	Job	Key fact
`fork()`	Clone the current process	Parent and child both continue running; only the return value differs.
`execvp()`	Replace a process image with a new program	The 'p' means it searches PATH when the name has no '/'.
`waitpid()`	Parent waits for / reaps a child	Also supports WNOHANG for background job management.

2 · Feature Scope

Three tiers. Level 1 and 2 are the evaluation floor — if either is incomplete on Oct 12, that is the first thing to fix, before any Level 3 polish.

LEVEL 1 — Baseline shell — must work

- Interactive prompt (`csh>`)
- External commands: `ls, pwd, cat, echo, ps` (and anything else on PATH)
- Command arguments: `ls -l`, `cat file.txt`
- Built-ins: `cd`, `pwd`, `exit`

LEVEL 2 — Core systems — what makes this a systems project

- Input redirection <, output >, append >>, stderr 2>
- Single pipe: `ls | grep ".c"`
- Background execution: `sleep 20 &`
- SIGINT (Ctrl+C) handling that keeps the shell alive

LEVEL 3 — Stretch, in priority order — only after Level 1 & 2 are stable

- **1.** `history` command (in-memory)
- **2.** Multiple pipelines: `cat f | grep x | wc -l`
- **3.** Combined redirection: `cmd > out 2> err`
- **4.** Quoted arguments: `echo "Hello Handsome"`
- **5.** stderr→stdout merge: `cmd > out 2>&1`
- **6.** Persistent history across sessions
- **7.** Environment variable expansion (`$HOME`)
- **8.** Simple autocomplete

Ranked by evaluation value per hour of effort. If Oct 9 arrives and you're not through item 2, stop — items 5–8 are not worth the risk to Level 1/2 stability.

Explicitly out of scope

Bash clone · sudo implementation · authentication / user management · GUI · themes / animations · a shell scripting language. None of these teach the OS concepts this project targets — they cost time without evaluation value.

3 · Risk Map & Cut-Line

Decide the cut-line now, while calm, not on Oct 10 under pressure. If you're behind schedule at any checkpoint below, apply the listed cut and move on — don't renegotiate scope mid-panic.

Checkpoint	If you're behind here...	Cut this first
End of Day 10 (pipes)	Single pipe still not reliable	Drop multiple pipelines (Day 11) entirely — go straight to background (Day 12)
End of Day 13 (signals)	Ctrl+C still kills the whole shell	Ship with documented known-issue; do not sink >1 extra day into signal edge cases
End of Day 15 (parser)	Combined syntax (pipe + redirect) breaks	Support pipe-only and redirect-only as separate cases; document the gap
Day 16 (buffer)	Still catching up on Level 2 items	Skip all remaining Level 3 stretch — Level 1+2 correctness beats stretch breadth

Why Day 16 exists

The original 19-day plan had zero slack. Pipes (Day 9–11) and signals (Day 13) are the two places almost every student implementation stalls — usually on a hanging pipe from an unclosed file descriptor, or a signal handler that kills the shell along with the child. Day 16 is reserved specifically to absorb that overflow before integration testing begins.

4 · Daily Plan at a Glance

#	Date	Focus	Level
1	Sep 23	Read loop: prompt, fgets, echo input	L1
2	Sep 24	First fork() + execvp() — run external commands	L1
3	Sep 25	argv parsing — arbitrary arguments	L1
4	Sep 26	Built-ins: cd, pwd, exit	L1
5	Sep 27	Refactor into modules (parser / builtin / main)	—
6	Sep 28	File descriptors — study day, tiny experiments only	L2
7	Sep 29	Output redirection > and >>	L2
8	Sep 30	stderr redirection 2>	L2
9	Oct 1	Pipes — study + standalone pipe() experiment only	L2
10	Oct 2	Implement one pipe: ls | grep	L2
11	Oct 3	Multiple pipelines (A | B | C) — stretch if tight	L3
12	Oct 4	Background execution with &	L2
13	Oct 5	SIGINT handling	L2
14	Oct 6	history built-in	L3
15	Oct 7	Parser upgrade for combined syntax	L2/L3
16	Oct 8	BUFFER — catch-up day, no new features by default	—
17	Oct 9	Memory + error handling audit across every syscall	L1/L2
18	Oct 10	Documentation + architecture diagrams	—
19	Oct 11	Final rehearsal — 6 demos, explain without README	—

5 · Day-by-Day Guide

Each day: Study First → Build → Checkpoint. The checkpoint questions are the actual evaluation prep — answer them out loud, not just mentally.

DAY 1 Sep 23

Build the shell that can read input

STUDY FIRST

— main(), printf(), fgets(), stdin/stdout
— How strings and newlines behave
— Do NOT study fork() yet — this day is input only

BUILD

```
csh> hello
You entered: hello

csh> ls
You entered: ls
```

CHECKPOINT — answer out loud

? Where does the typed string live in memory?
? What happens when the user presses Enter?
? How does the loop know to ask again?

far away
ROM → immutable
fgets → take stdin → store to buffer
↓
print buffer
↳ while(true)
↳ infinite loop

DAY 2 Sep 24

Execute your first program

STUDY FIRST

— process, fork(), exec(), execvp(), waitpid()
— PID, parent vs. child
— GNU C Library process chapter (see §8)

```
      fork()
       |
  +----+----+
  |         |
child     parent
  |         |
exec()   waitpid()
  |
 ls
```

BUILD

```
csh> ls
csh> pwd
csh> date
csh> whoami
```

CHECKPOINT — answer out loud

? Why can't I exec() directly inside the shell? our shell will close off
? What happens to the child after execvp() succeeds? it exits and parent get it
? Why does the parent need waitpid()? so know when child exits

Rule: Do not use system() to run commands. The whole point of this day is understanding process creation yourself.

DAY 3 Sep 25

Arguments and basic parsing

STUDY FIRST

— argc / argv, char**, NULL-terminated arrays

— String tokenization basics

BUILD

```
ls -l /home
  -> argv[0] = "ls"
     argv[1] = "-l"
     argv[2] = "/home"
     argv[3] = NULL

csh> ls -l
csh> cat file.txt
csh> ps aux
csh> echo hello
```

CHECKPOINT — answer out loud

? Why does execvp() take an array instead of one string?

? Does my shell still special-case 'ls', or does it now run anything + args?

DAY 4 Sep 26

Built-in commands

STUDY FIRST

— The distinction between an external command and a shell built-in

BUILD

```
csh> pwd
/home/rishi/c-shell
csh> cd ..
csh> pwd
/home/rishi
```

This is one of the strongest evaluation questions in the whole project — be ready to answer it without notes.

CHECKPOINT — answer out loud

? Why can't cd just be fork() + exec("cd")? → cause child and parent have different memory space

? What part of the process model explains this? (Hint: a child's working-directory change never propagates to its parent.)

the kernel keeps the working dir per process
fork gives child it's own copy

DAY 5 Sep 27

Clean up your architecture

STUDY FIRST

— header files, function declarations, translation units

— gcc compilation basics, Makefile basics

BUILD

```
src/
+-- main.c
+-- parser.c / parser.h
+-- builtin.c / builtin.h

$ make
$ make clean
```

No huge feature today — this is a refactor day only. You decide the exact module split; don't force a structure just because it's listed here.

CHECKPOINT — answer out loud

? Why did I separate these specific functions?

? Does this structure still make sense once file descriptors and pipes arrive in the next few days — or will I need to revisit it?

DAY 6 Sep 28

File descriptors — the most important study day

STUDY FIRST

— 0 = stdin, 1 = stdout, 2 = stderr

— open(), close(), read(), write(), dup2()

— A child inherits copies of its parent's file descriptors after fork()

BUILD

```
// Tiny experiments only — no shell code today.
program -> stdout -> terminal
program -> stdout -> file   (after you redirect fd 1)
```

Don't touch the shell's source today. dup2() is the single mechanism behind all of tomorrow's and the next day's redirection work — get it solid first.

CHECKPOINT — answer out loud

? What is a file descriptor, precisely?

? Why is stdout fd 1 and not, say, 0?

? What exactly does dup2(oldfd, newfd) change?

1. file descriptor is non negative number that process uses to refer to an open file (pipe, or socket, or terminal

2. fd 0 → stdin
   fd 1 → stdout
   fd 2 → stderror

3. it newfd pointing to something else first it get repointed to oldfd while oldfd remain untouched.

+1 → Kernel Keeps a table, per process, mapping each of these integer to an actual open-file entry. The integer itself have no meaning outside the process — fds in shell & fds in ls (unrelated)

DAY 7 Sep 29

Output redirection

STUDY FIRST

— open() flags: O_WRONLY, O_CREAT, O_TRUNC, O_APPEND

— dup2() + close() sequencing

```
before: ls -> stdout(1) -> terminal
after:  ls -> stdout(1) -> output.txt   (via dup2 before execvp)
```

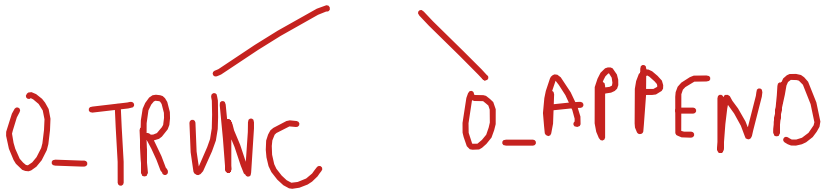
BUILD

```
csh> ls > output.txt
csh> echo hello > output.txt
csh> ls >> output.txt
```

CHECKPOINT — answer out loud

? Run: csh> ls > output.txt then csh> cat output.txt — does it match?

? What's the difference in open() flags between > and >>?



DAY 8 Sep 30 stderr redirection

STUDY FIRST

— stdout (1) and stderr (2) are separate streams

BUILD

```
csh> nonexistent_command 2> error.txt
csh> command > output.txt 2> error.txt
```

CHECKPOINT — answer out loud

? Deliberately cause an error — does it land in error.txt and NOT output.txt? Yes

? Can I explain why stdout and stderr need separate dup2() calls?

"dup2 changes one fd table entry per call, and stdout and stderr are 2 seperate entries, so each needs its own call"

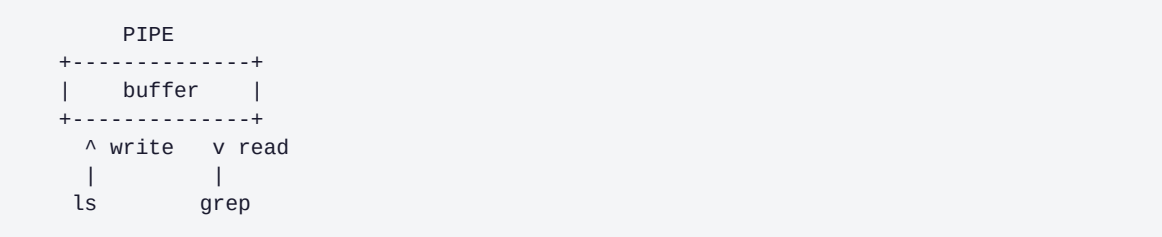
DAY 9 Oct 1

Pipes — understand before implementing

STUDY FIRST

— pipe(), read end vs. write end, EOF behavior

— Both processes inherit both ends after fork() — unused ends must be closed



BUILD

```
// Standalone experiment only — pipe() + fork(), not the shell.
```

CHECKPOINT — answer out loud

? Which process writes? Which reads?

? Why do both processes inherit the pipe's file descriptors?

? Why must the unused end be closed in each process?

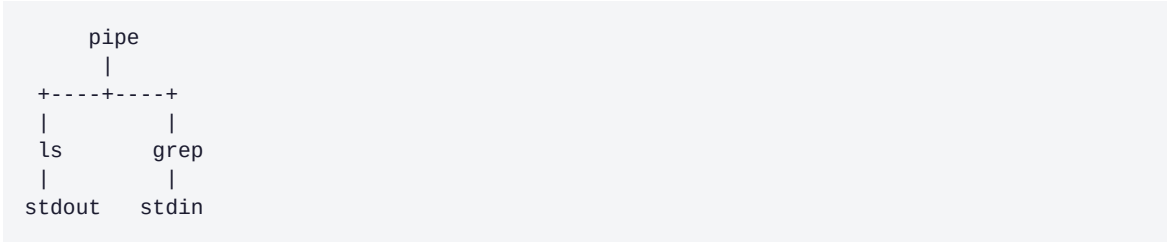
Rule: Do not implement the shell's pipe today. Build a tiny standalone C program with pipe() + fork() first, and understand why it works.

DAY 10 Oct 2

Implement one pipe

STUDY FIRST

— Combine pipe() + fork() + dup2() + close() + execvp() + waitpid()



BUILD

```
csh> ls | grep ".c"
csh> cat file.txt | grep error
```

CHECKPOINT — answer out loud

? Does the pipeline complete without hanging?

? If it hangs — have I closed every unused fd in every process, including both ends in the parent?



DAY 11 Oct 3

Multiple pipelines

STUDY FIRST

— A | B | C as: A.stdout -> pipe1 -> B.stdin, B.stdout -> pipe2 -> C.stdin

BUILD

```
csh> cat file.txt | grep error | wc -l
```

CHECKPOINT — answer out loud

? Does A | B | C work end to end?

? If this is taking too long — have I stopped at two commands and marked the rest as stretch, per the cut-line in Section 3?

DAY 12 Oct 4

Background processes

STUDY FIRST

— waitpid() with WNOHANG for non-blocking job management

BUILD

```
csh> sleep 10 &
csh>                <- returns immediately
```

CHECKPOINT — answer out loud

? Run sleep 20 & immediately followed by pwd — does the shell stay usable while sleep runs?

DAY 13 Oct 5

Signals

STUDY FIRST

— SIGINT, SIGCHLD, signal() vs. sigaction()

BUILD

```
csh> sleep 100
^C
csh>                <- shell survives, sleep is interrupted
```

CHECKPOINT — answer out loud

? Which process actually receives SIGINT in my implementation — the shell, the foreground child, or both?

? Does Ctrl+C against the shell itself (with no child running) behave sanely?

DAY 14 Oct 6

History

STUDY FIRST

— Pick a data structure deliberately: array, dynamic array, or linked list — don't default to one just because it's familiar

BUILD

```
csh> ls
csh> pwd
csh> cd ..
csh> history
1 ls
2 pwd
3 cd ..
4 history
```

CHECKPOINT — answer out loud

? What data structure did I choose, and why does it fit my requirements better than the alternatives?

DAY 15 Oct 7

Parser upgrade

STUDY FIRST

— Tokenization vs. parsing — start with a simple whitespace tokenizer, evolve it

BUILD

```
ls -l
ls > output.txt
ls | grep ".c"
ls | grep ".c" > output.txt
sleep 10 &
echo "Hello Handsome"   (stretch: quoting)
```

CHECKPOINT — answer out loud

? Before coding: what does my parser produce, step by step, for ls | grep ".c" > output.txt ?

DAY 16 Oct 8

BUFFER — catch-up day

STUDY FIRST

— No new concepts by default

— Review anything from Days 9–13 that felt shaky

BUILD

```
// No prescribed build target.
// Use this day to finish whatever from Level 2 is still incomplete.
// If everything is done: start early on Day 17's error-handling audit.
```

This day exists because pipes and signals are where most implementations slip. Use the Risk & Cut-Line table in Section 3 to decide what to drop if you're still behind at the end of today.

CHECKPOINT — answer out loud

? Am I fully caught up on Level 1 + Level 2? If not, what specifically is blocking me, and is it a concept gap or a bug?

DAY 17 Oct 9

Memory, error handling, cleanup

STUDY FIRST

— malloc/calloc/realloc/free, errno, perror, return-value checking

BUILD

```
// Audit every one of these call sites in your code:
malloc()   fork()   execvp()
pipe()     open()   dup2()   waitpid()

// Then intentionally trigger:
command doesn't exist
file doesn't exist / can't open
invalid syntax
```

CHECKPOINT — answer out loud

? For each syscall above: what happens if it fails, and did I check the return value?

? Does the shell survive all four intentional failures above without crashing?

DAY 18 Oct 10

Documentation + architecture

STUDY FIRST

— Write the README properly — this is graded material, not an afterthought

BUILD

```
README.md sections:
Project Description, Motivation, Goals, Non-Goals, Features,
Scope, Architecture, Design, Technologies/APIs, Testing,
Milestones, Expected Learning, Limitations

+ the two diagrams from Section 1, reproduced with real detail
  from your actual implementation.
```

CHECKPOINT — answer out loud

? Could someone who has never seen my code understand the architecture from the README alone?

DAY 19 Oct 11

Final evaluation rehearsal

STUDY FIRST

— No new features today unless something is actively broken

BUILD

```
Demo 1 — basic:      pwd, ls, ls -l, echo "Hello Handsome"
Demo 2 — built-ins:  cd .., pwd, history
Demo 3 — redirect:   ls > output.txt && cat output.txt
Demo 4 — pipeline:   ls | grep ".c"
Demo 5 — background: sleep 10 &  then  pwd
Demo 6 — signals:    sleep 100  then Ctrl+C
```

CHECKPOINT — answer out loud

? Can I run all six demos from a clean checkout without notes?

? Can I explain the architecture out loud without opening the README?

Rule: Run the project from a completely clean checkout before the rehearsal — not your working directory. This catches missing files and undocumented environment assumptions.

6 · AI Collaboration Rules

This section exists so the project stays defensible on Oct 12. The rule is simple: **you implement, Claude teaches, reviews, and debugs.**

YOU MAY ASK	DON'T ASK
✓ What does fork() do? ✓ Why does execvp() replace the process image? ✓ Why does cd need to be a built-in? ✓ Explain file descriptors / dup2(). ✓ Why is my pipe hanging? ✓ What does this compiler error mean? ✓ Review my parser design or my code for bugs. ✓ Give me test cases or evaluation questions to rehearse.	✗ Build the whole shell. ✗ Give me the complete parser. ✗ Give me the complete pipe implementation. ✗ Give me the final project. ✗ Write all the code for Day 10 (or any day).

If you're genuinely stuck, hints escalate in this order:

```
Hint 1 -> Concept -> Relevant API -> Pseudocode -> Debug your attempt
   -> (only if still stuck) implementation guidance
```

7 · Debugging Protocol

When something breaks, work through this before asking for help:

```
1. What did I expect?
2. What actually happened?
3. Where does the behavior first become different?
4. What values do I expect?
5. What values do I actually have?
6. What does the relevant API/documentation say?
7. Can I make a tiny program that reproduces the problem?
```

Then bring exactly this to the conversation:

```
Expected:  ...
Actual:    ...
My reasoning: ...
Code:      ...
Error/output: ...
```

8 · Reference Library

One hierarchy, not fifty tutorials.

```
MASTER GUIDE (this document)
      |
      +-- Study concept
      |
      +-- GNU C Library Reference Manual (process chapter)
      |
      +-- Linux man-pages for the exact API
      |
      +-- Build + experiment
```

fork(2)	man7.org/linux/man-pages/man2/fork.2.html
exec(3) / execvp	man7.org/linux/man-pages/man3/execlp.3.html
wait(3p) / waitpid	man7.org/linux/man-pages/man3/wait.3p.html
open(2)	man7.org/linux/man-pages/man2/open.2.html
dup2(2)	man7.org/linux/man-pages/man2/dup2.2.html
pipe(2) / pipe(7)	man7.org/linux/man-pages/man2/pipe.2.html
GNU libc — process chapter	gnu.org/software/libc/manual (ch. 26, Processes)

Appendix A · Learning Notebook Template

Keep this as `LEARNING.md` in your repo. Fill one block per concept as you build it — fork(), execvp(), waitpid(), file descriptors, dup2(), open(), pipe(), signals, parsing, background processes.

```
## fork()

What I thought before:
...

What I learned:
...

Why does the shell need it?
...

What confused me:
...

How my implementation uses it:
...
```

Appendix B · Full Integration Test Matrix

Run this on Day 16 (buffer) and again on Day 19 (rehearsal).

Basic	■ pwd ■ ls ■ ls -l ■ cat file
Built-ins	■ cd ■ pwd ■ exit ■ history
Redirection	■ > ■ >> ■ < ■ 2>
Pipes	■ A | B ■ A | B | C
Background	■ command &
Signals	■ Ctrl+C
Errors	■ nonexistent command ■ nonexistent file ■ malformed pipe ■ malformed redirection

Appendix C · Pre-Evaluation Self-Test Script

A simple shell script you can run on Day 19 to make the rehearsal objective instead of a vibe check. Adapt paths/commands to your actual build output.

```
#!/bin/bash
# selftest.sh — run from a clean checkout, after `make`
set -u
PASS=0; FAIL=0
check() {
  echo "--- $1 ---"
  echo "$2" | ./csh > /tmp/csh_out.txt 2>&1
  if grep -q "$3" /tmp/csh_out.txt; then
    echo "PASS"; PASS=$((PASS+1))
  else
    echo "FAIL (expected to see: $3)"; FAIL=$((FAIL+1))
  fi
}

check "pwd works"        "pwd"                "/"
check "ls works"         "ls"                 "."
check "cd + pwd"         "cd .. && pwd"       "/"
check "redirection"      "ls > /tmp/o.txt"    ""
check "pipe"             "ls | grep c"        "c"
check "background"       "sleep 1 &"          ""

echo "============================"
echo "PASS: $PASS   FAIL: $FAIL"
```

This is a starting skeleton, not a complete suite — expand the checks to cover everything in your Level 1 and Level 2 scope before Oct 11.

The goal on Oct 12 isn't "I built a shell." It's: "I didn't understand this, I hit a problem building it, I studied the mechanism, implemented it, broke it, debugged it — and here's why my implementation works." That sentence is the actual deliverable.